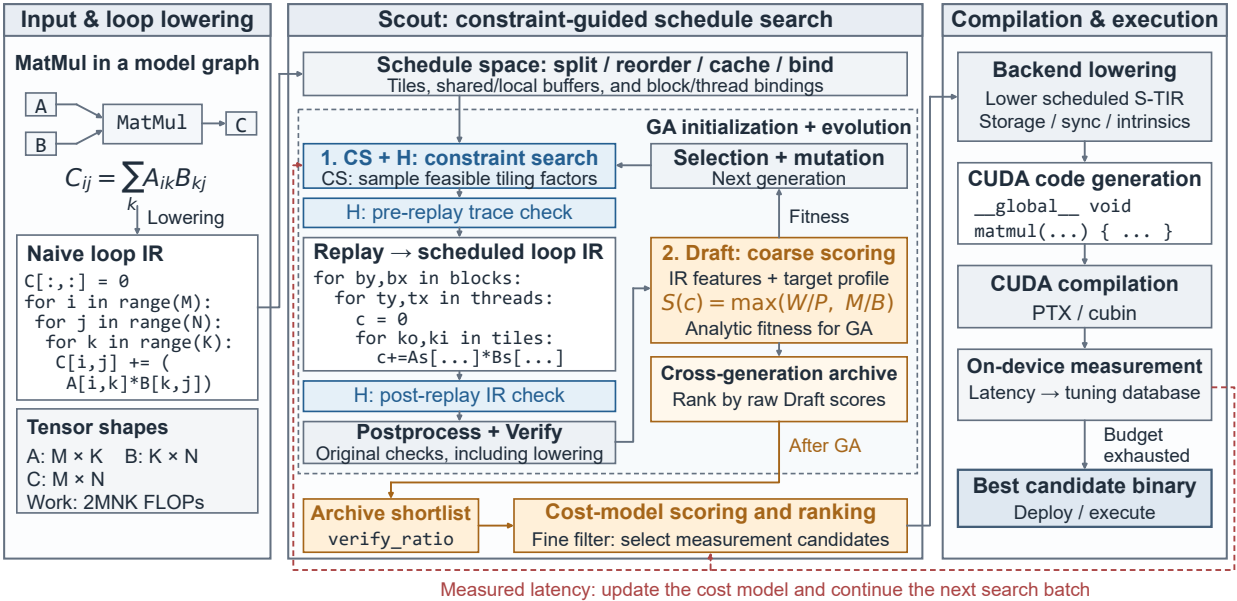

Input & loop lowering
MatMul in a model graph
A
B
MatMul
C
$C_{ij} = \sum_k A_{ik} B_{kj}$
Lowering
Naive loop IR
C[:,:] = 0
for i in range(M):
for j in range(N):
for k in range(K):
C[i,j] += (
A[i,k]*B[k,j])
Tensor shapes
A: M × K B: K × N
C: M × N
Work: 2MNK FLOPs
Scout: constraint-guided schedule search
Schedule space: split / reorder / cache / bind
Tiles, shared/local buffers, and block/thread bindings
GA initialization + evolution
1. CS + H: constraint search
CS: sample feasible tiling factors
Selection + mutation
Next generation
H: pre-replay trace check
Fitness
Replay → scheduled loop IR
for by,bx in blocks:
for ty,tx in threads:
c = 0
for ko,ki in tiles:
c+=As[...]*Bs[...]
2. Draft: coarse scoring
IR features + target profile
$S(c) = \max(W/P,\ M/B)$
Analytic fitness for GA
H: post-replay IR check
Cross-generation archive
Rank by raw Draft scores
Postprocess + Verify
Original checks, including lowering
After GA
Archive shortlist
verify_ratio
Cost-model scoring and ranking
Fine filter: select measurement candidates
Compilation & execution
Backend lowering
Lower scheduled S-TIR
Storage / sync / intrinsics
CUDA code generation
__global__ void
matmul(...) { ... }
CUDA compilation
PTX / cubin
On-device measurement
Latency → tuning database
Budget exhausted
Best candidate binary
Deploy / execute
Measured latency: update the cost model and continue the next search batch